ORANGE GROCERY UI

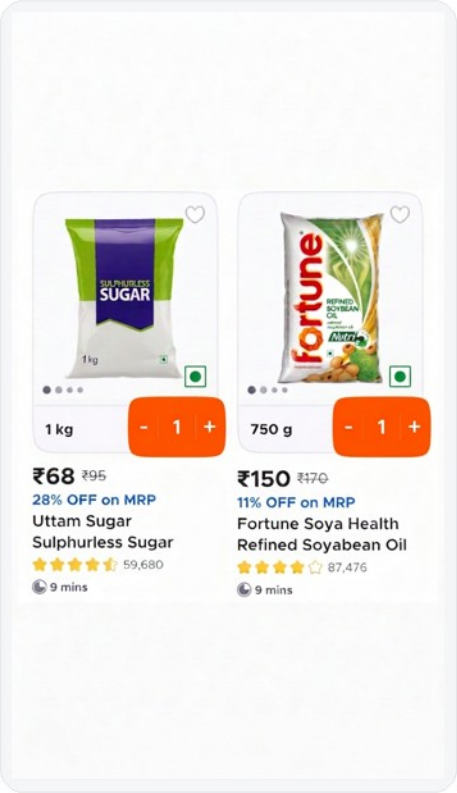
Complete code + interaction specification

A standalone, responsive vanilla HTML / CSS / JavaScript implementation

ORANGE | VEG BADGE | ACCESSIBLE

What this PDF contains

- A visual card design system using orange as the purchase-control colour.
- Standard vegetarian product indicator that intentionally remains green.
- Direct ADD → orange − 1 + quantity stepper behavior for one-option products.
- ADD → bottom-sheet selector → Add to cart behavior for multi-option products.
- Separate cart line items for every selected product variant.
- The entire runnable source code in one HTML file, reproduced in the appendix.

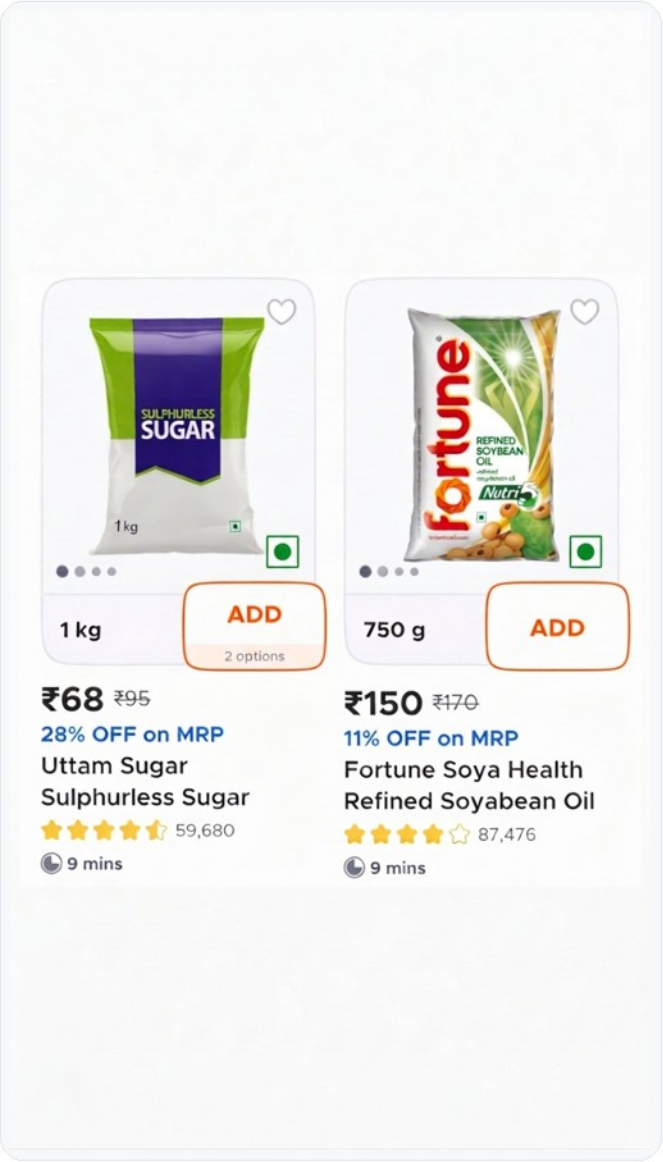


Visual reference: orange CTA and quantity stepper; vegetarian badge stays green.

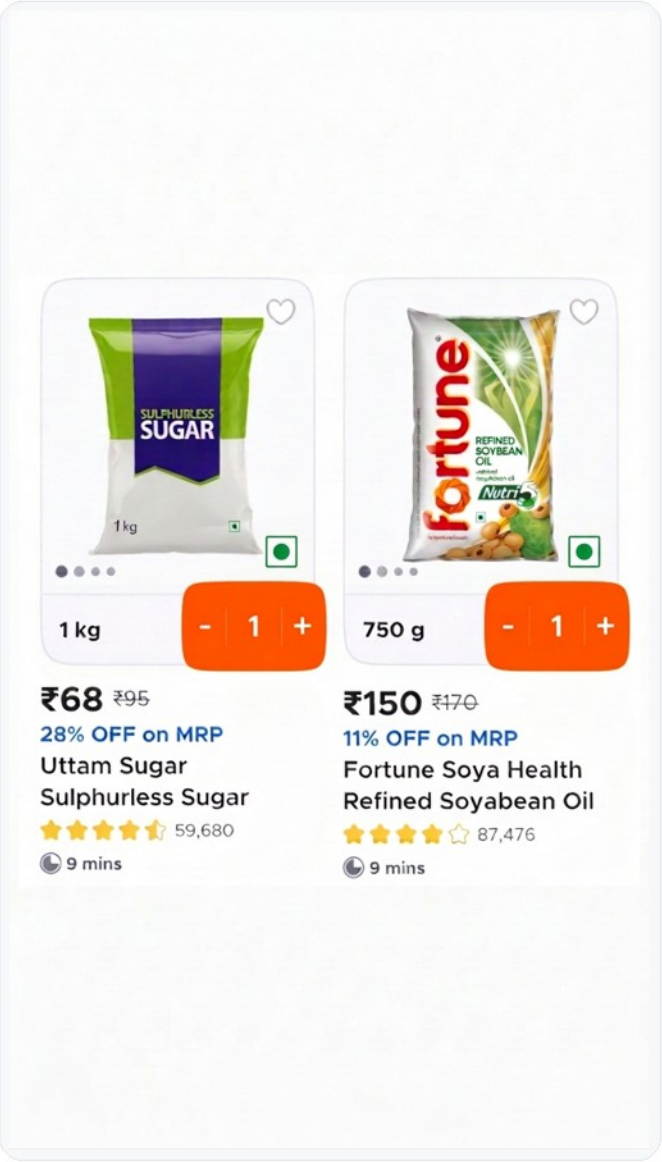
1 Attached visual states

Both orange UI variants used by the code are embedded below

The implementation supports both of these states: the outlined ADD state for a multi-option product, and the filled orange quantity stepper after an item is added. The vegetarian badges remain green in both images.



A. Multi-option initial state — tap ADD to open the variant sheet.



B. Added state — orange − 1 + quantity stepper.

1 Run the project

One file, no framework or external asset dependency

Save the provided source as grocery-orange-ui-demo.html. Open it in any modern desktop or mobile browser. The sample uses CSS-built product packages so the demo works offline; replace the renderPackage output with your own optimized product images when connecting to a real catalog.

Files delivered

grocery-orange-ui-demo.html is the interactive implementation. orange-grocery-ui-code-guide.pdf is this specification and full-source appendix.

No install step

The demo uses browser-native HTML, CSS and JavaScript. There is no npm command, library import, build process or API key required to test the interactions.

Quick test checklist

- Click the oil ADD button: it changes directly to the orange − 1 + quantity control.
- Click the sugar ADD button: a Select an option bottom sheet opens.
- Choose 1 kg or 2 kg, then click Add to cart: the sheet closes and that selected size becomes the active card variant.
- Use Change option to add a different variant; it remains a separate cart Map entry.

2 Visual system

Card geometry, colour rules and design tokens

Core colour rules

- #FF5A00 — ADD / stepper / selected option
- #237A35 — vegetarian food badge only
- #2F6FC9 — discount copy
- #20252F — primary text

Product card

20 px outer radius, 1 px light-grey border, pale image canvas, product image, favourite icon, carousel dots and green vegetarian indicator. The footer contains the selected variant and CTA.

Overlap rule

The ADD or − 1 + control sits above the footer divider using position: absolute. It overlaps the lower-right card boundary by a small amount while remaining legible and touchable.

Control rule

Orange is reserved for actionable purchase controls. The vegetarian mark remains green so it continues to carry its established product-category meaning.

3 Interaction behavior

Exact ADD, options, cart and quantity rules

1. **One-option product**
   Tap ADD. Add one unit immediately, display a toast, update the cart counter and replace ADD with the orange − 1 + stepper.

2. **Multi-option product**
   Tap ADD. Do not add to the cart yet. Open a bottom sheet showing product name and every available option with quantity, price, MRP, discount and stock state.

3. **Variant selection**
   The selected option uses an orange radio indicator and border. The user taps Add to cart to confirm; tapping the scrim, close icon or Escape closes without changing the cart.

4. **Selected state**
   After confirmation, close the sheet. Display the selected size in the footer and show the orange − 1 + stepper for that variant.

5. **Separate variants**
   The cart key is productId:variantId. Therefore 1 kg and 2 kg quantities are independent line items. Change option opens the same sheet so another size can be added.

6. **Quantity limits**
   Minus decrements the active variant. At zero, remove that cart line and return its card control to ADD or ADD + options.

4 Production notes

Data contract, accessibility and integration points

Catalog data

Each product has id, name, reviews, presentation data and options[]. Each option has id, label, price, mrp, discount and availability. Replace the hardcoded products array with API data.

Cart store

This demo uses a JavaScript Map to stay framework-free. In an app, persist the same productId:variantId key and quantity in your cart store, local storage or backend API.

Accessibility

The options panel is a labelled modal dialog. Radio controls are native and keyboard usable. Escape and the scrim close the sheet. Buttons contain accessible labels and the cart count is live.

Images

Replace the CSS package illustrations with <img loading="lazy" alt="..."> for real product thumbnails. Keep object-fit: contain and preserve image-area padding.

Backend safety

Always re-check price, availability, cart limits and variant stock on the server before checkout. The UI is a client-side representation only.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Orange Grocery Product Cards</title>
  <style>
    :root {
      --orange: #FF5A00;
      --orange-dark: #E64E00;
      --orange-soft: #FFF0E8;
      --green: #237A35;
      --green-dark: #176128;
      --ink: #20252E;
      --muted: #697180;
      --line: #E3E6EB;
      --surface: #FFFFFF;
      --subtle: #F6F7F9;
      --blue: #2F6FC9;
      --star: #FFC93D;
      --radius-card: 20px;
      --radius-control: 16px;
      --shadow: 0 12px 32px rgba(28, 37, 50, .12);
    }

    * { box-sizing: border-box; }
    body {
      margin: 0;
      min-width: 320px;
      color: var(--ink);
      background: #F4F6F8;
      font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont,
        "Segoe UI", sans-serif;
    }
    button, input { font: inherit; }
    button { cursor: pointer; }
    button:focus-visible, input:focus-visible {
      outline: 3px solid rgba(255, 90, 0, .32);
      outline-offset: 3px;
    }

    .app {
      max-width: 840px;
      min-height: 100vh;
      margin: 0 auto;
      padding: 22px 18px 56px;
      background: var(--surface);
    }
    .topbar {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 16px;
      margin-bottom: 22px;
    }
    .eyebrow {
      margin: 0 0 4px;
      color: var(--orange-dark);
      font-size: 12px;
      font-weight: 800;
      letter-spacing: .09em;
      text-transform: uppercase;
    }
    h1 {
      margin: 0;
      font-size: clamp(21px, 4vw, 30px);
      letter-spacing: -.04em;
      line-height: 1.08;
    }
    .cart-summary {
      display: flex;
      align-items: center;
      gap: 8px;
      border: 1px solid var(--line);
      border-radius: 999px;
      padding: 8px 12px;
      background: #fff;
      font-size: 13px;
      font-weight: 760;
      white-space: nowrap;
    }
    .cart-bubble {
      display: grid;
      width: 21px;
      height: 21px;
      place-items: center;
      border-radius: 50%;
      color: #fff;
      background: var(--orange);
      font-size: 12px;
    }

    .product-grid {
      display: grid;
      grid-template-columns: repeat(2, minmax(0, 1fr));
      gap: clamp(16px, 3vw, 30px);
    }
    .listing { min-width: 0; }
    .visual-card {
      position: relative;
      overflow: visible;
      border: 1px solid var(--line);
```

```
      border-radius: var(--radius-card);
      background: #fff;
    }
    .image-area {
      position: relative;
      height: clamp(220px, 37vw, 320px);
      overflow: hidden;
      border-radius: var(--radius-card) var(--radius-card) 0 0;
      background:
        radial-gradient(circle at 54% 35%, rgba(255,255,255,.9), transparent 42%),
        linear-gradient(145deg, #FAFBFC, #EFF1F4);
    }
    .wish {
      position: absolute;
      z-index: 2;
      top: 13px;
      right: 13px;
      display: grid;
      width: 34px;
      height: 34px;
      place-items: center;
      border: 0;
      padding: 0;
      color: #89919C;
      background: transparent;
      font-size: 29px;
      line-height: 1;
    }
    .wish.is-favourite { color: #EC4C54; }
    .pack-wrap {
      position: absolute;
      inset: 20px 26px 29px;
      display: grid;
      place-items: center;
    }
    .package {
      position: relative;
      display: flex;
      flex-direction: column;
      justify-content: center;
      align-items: center;
      width: min(74%, 190px);
      height: min(90%, 220px);
      overflow: hidden;
      border-radius: 14px 15px 20px 17px;
      box-shadow: 0 16px 22px rgba(26, 32, 42, .17);
      color: white;
      text-align: center;
      transform: rotate(-2deg);
    }
    .package::before {
      position: absolute;
      content: "";
      inset: -45% 44% -20% -38%;
      background: rgba(255,255,255,.28);
      transform: rotate(22deg);
    }
    .package::after {
      position: absolute;
      top: 0;
      right: 0;
      bottom: 0;
      width: 8px;
      background: rgba(0,0,0,.1);
    }
    .package.sugar {
      background: linear-gradient(90deg, #73BC13 0 28%, #2F2377 28% 73%, #7ABD16 73%);
    }
    .package.oil {
      background: linear-gradient(135deg, #FFFFFF 0 31%, #D9EBA7 31% 35%, #3E8A39 35% 81%, #F0B82B 81%);
      color: #28723A;
      transform: rotate(2deg);
    }
    .pack-brand, .pack-name, .pack-copy, .pack-qty { position: relative; z-index: 1; }
    .pack-brand {
      margin-bottom: 14px;
      font-size: clamp(11px, 2.1vw, 16px);
      font-weight: 900;
      letter-spacing: .08em;
      text-transform: uppercase;
    }
    .pack-name {
      max-width: 85%;
      font-size: clamp(21px, 4vw, 34px);
      font-weight: 900;
      letter-spacing: -.07em;
      line-height: .82;
    }
    .oil .pack-name {
      color: var(--orange);
      font-size: clamp(25px, 4.6vw, 42px);
      transform: rotate(-90deg);
    }
    .pack-copy { margin-top: 15px; font-size: 9px; font-weight: 700; }
    .pack-qty { position: absolute; bottom: 14px; left: 15px; font-size: 14px; font-weight: 800; }

    .dots {
      position: absolute;
      bottom: 17px;
      left: 17px;
      display: flex;
      gap: 7px;
```

```
      }
      .dot { width: 11px; height: 11px; border-radius: 50%; background: #BDC3CB; }
      .dot.active { background: #404650; }
      .veg-badge {
        position: absolute;
        right: 17px;
        bottom: 16px;
        display: grid;
        width: 32px;
        height: 32px;
        place-items: center;
        border: 2px solid var(--green);
        border-radius: 4px;
        background: #fff;
      }
      .veg-badge::after {
        width: 17px;
        height: 17px;
        content: "";
        border-radius: 50%;
        background: var(--green);
      }
      .variant-footer {
        position: relative;
        min-height: 81px;
        display: flex;
        align-items: center;
        padding: 21px 17px 11px;
        border-top: 1px solid var(--line);
        border-radius: 0 0 var(--radius-card) var(--radius-card);
        background: #F8F9FB;
      }
      .variant-label { font-size: clamp(17px, 3vw, 23px); font-weight: 800; letter-spacing: -.04em; }

      .add-control, .stepper-wrap {
        position: absolute;
        z-index: 3;
        right: -10px;
        top: -19px;
        width: min(57%, 205px);
      }
      .add-button {
        display: grid;
        width: 100%;
        min-height: 73px;
        place-items: center;
        border: 2px solid var(--orange);
        border-radius: var(--radius-control);
        color: var(--orange-dark);
        background: #fff;
        font-size: clamp(18px, 3vw, 25px);
        font-weight: 850;
        letter-spacing: -.035em;
      }
      .add-button small { display: block; margin-top: -8px; color: #87909C; font-size: 11px; font-weight: 700; letter-spacing: 0; }
      .stepper {
        display: grid;
        grid-template-columns: 1fr 1fr 1fr;
        min-height: 64px;
        overflow: hidden;
        border: 1px solid var(--orange-dark);
        border-radius: var(--radius-control);
        color: #fff;
        background: var(--orange);
        box-shadow: 0 7px 17px rgba(255,90,0,.18);
      }
      .stepper button, .stepper output {
        display: grid;
        min-width: 0;
        place-items: center;
        border: 0;
        color: inherit;
        background: transparent;
        font-size: 25px;
        font-weight: 720;
      }
      .stepper button:hover { background: rgba(0,0,0,.08); }
      .stepper output { border-right: 1px solid rgba(255,255,255,.35); border-left: 1px solid rgba(255,255,255,.35); font-size: 17px; font-weight: 850; }
      .option-link {
        display: block;
        width: 100%;
        margin: 7px auto 0;
        border: 0;
        color: var(--orange-dark);
        background: transparent;
        font-size: 12px;
        font-weight: 800;
        text-decoration: underline;
        text-underline-offset: 3px;
      }

      .product-copy { padding: 19px 0 4px; }
      .price-line { display: flex; align-items: baseline; gap: 8px; flex-wrap: wrap; }
      .price { font-size: clamp(24px, 4vw, 35px); font-weight: 900; letter-spacing: -.055em; }
      .mrp { color: #747C88; font-size: 16px; text-decoration: line-through; }
      .discount { margin: 4px 0 8px; color: var(--blue); font-size: clamp(15px, 2.8vw, 21px); font-weight: 850; letter-spacing: -.035em; }
      .name { min-height: 2.65em; margin: 0; font-size: clamp(16px, 3vw, 23px); font-weight: 780; letter-spacing: -.04em; line-height: 1.32; }
      .rating, .delivery { display: flex; align-items: center; gap: 6px; margin-top: 11px; color: #68717E; font-size: 14px; font-weight: 650; }
      .stars { color: var(--star); font-size: 23px; letter-spacing: 1px; line-height: 1; }
      .clock { display: inline-grid; width: 21px; height: 21px; place-items: center; border: 2px solid #8B94A2; border-radius: 50%; color: #8B94A2;
    font-size: 13px; }

```

```
      .scrim { position: fixed; z-index: 20; inset: 0; display: none; background: rgba(18, 25, 36, .45); }
      .scrim.is-open { display: block; }
      .sheet {
        position: fixed;
        z-index: 21;
        right: 0;
        bottom: 0;
        left: 0;
        width: min(680px, 100%);
        max-height: 87vh;
        margin: 0 auto;
        overflow: auto;
        border-radius: 25px 25px 0 0;
        background: #fff;
        box-shadow: 0 -16px 42px rgba(22, 28, 38, .22);
        transform: translateY(105%);
        transition: transform .24s ease;
      }
      .sheet.is-open { transform: translateY(0); }
      .sheet-body { padding: 13px 20px max(24px, env(safe-area-inset-bottom)); }
      .sheet-handle { width: 44px; height: 5px; margin: 0 auto 17px; border-radius: 99px; background: #D5D9DF; }
      .sheet-header { display: flex; align-items: flex-start; justify-content: space-between; gap: 15px; margin-bottom: 18px; }
      .sheet-title { margin: 0; font-size: 23px; letter-spacing: -.045em; }
      .sheet-product { margin: 4px 0 0; color: var(--muted); font-size: 14px; font-weight: 650; }
      .close-sheet { display: grid; width: 34px; height: 34px; place-items: center; border: 1px solid var(--line); border-radius: 50%; color: #5D6570; background: white; font-size: 22px; }
      .option-list { display: grid; gap: 11px; }
      .option-row { position: relative; display: grid; grid-template-columns: auto 1fr auto; align-items: center; gap: 12px; padding: 15px; border: 1px solid var(--line); border-radius: 14px; background: #fff; cursor: pointer; }
      .option-row:has(input:checked) { border-color: var(--orange); background: var(--orange-soft); box-shadow: 0 0 0 1px var(--orange); }
      .option-row input { width: 19px; height: 19px; accent-color: var(--orange); }
      .option-size { display: block; margin-bottom: 4px; font-size: 17px; font-weight: 850; }
      .option-meta { color: var(--muted); font-size: 13px; font-weight: 600; }
      .option-price { text-align: right; font-size: 18px; font-weight: 900; }
      .option-price del { display: block; margin-top: 3px; color: var(--muted); font-size: 12px; font-weight: 500; }
      .sheet-add { width: 100%; margin-top: 17px; padding: 16px; border: 0; border-radius: 14px; color: white; background: var(--orange); font-size: 17px; font-weight: 850; }
      .sheet-add:hover { background: var(--orange-dark); }
      .toast { position: fixed; z-index: 30; bottom: 22px; left: 50%; padding: 12px 16px; border-radius: 999px; color: #fff; background: #20252E; box-shadow: var(--shadow); font-size: 14px; font-weight: 720; transform: translate(-50%, 115px); transition: transform .2s ease; white-space: nowrap; }
      .toast.is-visible { transform: translate(-50%, 0); }

      @media (max-width: 500px) {
        .app { padding: 17px 10px 38px; }
        .topbar { margin-bottom: 16px; }
        .product-grid { gap: 12px; }
        .image-area { height: 220px; }
        .pack-wrap { inset: 22px 18px 28px; }
        .variant-footer { min-height: 78px; padding: 23px 12px 10px; }
        .add-control, .stepper-wrap { right: -7px; width: 68%; top: -18px; }
        .add-button { min-height: 66px; }
        .stepper { min-height: 58px; }
        .price { font-size: 28px; }
        .name { font-size: 17px; }
        .stars { font-size: 19px; }
        .rating, .delivery { font-size: 12px; }
      }
    </style>
  </head>
  <body>
    <main class="app">
      <header class="topbar">
        <div>
          <p class="eyebrow">Fresh picks</p>
          <h1>Everyday groceries</h1>
        </div>
        <div class="cart-summary" aria-live="polite"><span>Cart</span><span id="cartCount" class="cart-bubble">0</span></div>
      </header>
      <section id="productGrid" class="product-grid" aria-label="Products"></section>
    </main>

    <div id="scrim" class="scrim" aria-hidden="true"></div>
    <section id="optionSheet" class="sheet" role="dialog" aria-modal="true" aria-labelledby="sheetTitle" aria-hidden="true"></section>
    <div id="toast" class="toast" role="status" aria-live="polite"></div>

    <script>
      /* Product data. Replace names, prices, image treatment, and variants with API data in production. */
      const products = [
        {
          id: 'sugar',
          name: 'Uttam Sugar Sulphurless Sugar',
          packClass: 'sugar',
          brand: 'Uttam Sugar',
          packName: 'sulphurfree<br>sugar',
          reviews: '59,680',
          options: [
            { id: 'sugar-1kg', label: '1 kg', price: 68, mrp: 95, discount: '28% OFF on MRP', availability: 'In stock' },
            { id: 'sugar-2kg', label: '2 kg', price: 130, mrp: 190, discount: '31% OFF on MRP', availability: 'In stock' }
          ]
        },
        {
          id: 'oil',
          name: 'Fortune Soya Health Refined Soyabean Oil',
          packClass: 'oil',
          brand: 'Soya health',
          packName: 'fortune',
          reviews: '87,476',
          options: [
            { id: 'oil-750g', label: '750 g', price: 150, mrp: 170, discount: '11% OFF on MRP', availability: 'In stock' }
          ]
        }
      ];
```

```

    const state = {
      // Cart is intentionally keyed by product + variant, so two variants become separate line items.
      cart: new Map(),
      activeVariantByProduct: {},
      selectedProductId: null,
      selectedVariantId: null
    };

    const productGrid = document.querySelector('#productGrid');
    const sheet = document.querySelector('#optionSheet');
    const scrim = document.querySelector('#scrim');
    const cartCount = document.querySelector('#cartCount');
    const toast = document.querySelector('#toast');

    function money(value) { return `₹${value}`; }
    function getProduct(productId) { return products.find(product => product.id === productId); }
    function getVariant(product, variantId) { return product.options.find(option => option.id === variantId); }
    function cartKey(productId, variantId) { return `${productId}:${variantId}`; }
    function quantity(productId, variantId) { return state.cart.get(cartKey(productId, variantId)) || 0; }
    function selectedVariant(product) {
      const fallback = product.options[0].id;
      return getVariant(product, state.activeVariantByProduct[product.id] || fallback);
    }
    function totalCartItems() {
      return [...state.cart.values()].reduce((sum, qty) => sum + qty, 0);
    }

    function renderPackage(product) {
      return `
        <div class="pack-wrap" aria-hidden="true">
          <div class="package ${product.packClass}">
            <span class="pack-brand">${product.brand}</span>
            <strong class="pack-name">${product.packName}</strong>
            <span class="pack-copy">Refined grocery essential</span>
            <span class="pack-qty">${product.options[0].label}</span>
          </div>
        </div>`;
    }

    function renderControl(product, option) {
      const currentQuantity = quantity(product.id, option.id);
      const hasMultipleOptions = product.options.length > 1;

      if (currentQuantity > 0) {
        return `
          <div class="stepper-wrap">
            <div class="stepper" aria-label="Quantity controls for ${product.name}">
              <button type="button" data-action="decrement" data-product-id="${product.id}" data-variant-id="${option.id}" aria-label="Decrease quantity">−</button>
              <output aria-live="polite">${currentQuantity}</output>
              <button type="button" data-action="increment" data-product-id="${product.id}" data-variant-id="${option.id}" aria-label="Increase quantity">+</button>
            </div>
            ${hasMultipleOptions ? `<button type="button" class="option-link" data-action="options" data-product-id="${product.id}">Change option</button>` : ''}
          </div>`;
      }

      return `
        <div class="add-control">
          <button type="button" class="add-button" data-action="${hasMultipleOptions ? 'options' : 'quick-add'}" data-product-id="${product.id}">
            <span>ADD</span>${hasMultipleOptions ? `<small>${product.options.length} options</small>` : ''}
          </button>
        </div>`;
    }

    function renderProducts() {
      productGrid.innerHTML = products.map(product => {
        const option = selectedVariant(product);
        return `
          <article class="listing">
            <div class="visual-card">
              <div class="image-area">
                <button type="button" class="wish" data-action="favourite" aria-label="Add ${product.name} to favourites">♡</button>
                ${renderPackage(product)}
                <div class="dots" aria-hidden="true"><span class="dot active"></span><span class="dot"></span><span class="dot"></span><span class="dot"></span></div>
                <span class="veg-badge" aria-label="Vegetarian product" role="img"></span>
              </div>
              <div class="variant-footer">
                <span class="variant-label">${option.label}</span>
                ${renderControl(product, option)}
              </div>
            </div>
            <div class="product-copy">
              <div class="price-line"><strong class="price">${money(option.price)}</strong><span class="mrp">${money(option.mrp)}</span></div>
              <p class="discount">${option.discount}</p>
              <h2 class="name">${product.name}</h2>
              <div class="rating"><span class="stars" aria-label="4.5 out of 5 stars">★★★★☆</span><span>${product.reviews}</span></div>
              <div class="delivery"><span class="clock" aria-hidden="true">◔</span><span>9 mins</span></div>
            </div>
          </article>`;
      }).join('');
      cartCount.textContent = totalCartItems();
    }

    function showToast(message) {
      toast.textContent = message;
      toast.classList.add('is-visible');
      window.clearTimeout(showToast.timer);
      showToast.timer = window.setTimeout(() => toast.classList.remove('is-visible'), 2100);
```

```
      }

      function setQuantity(productId, variantId, nextQuantity) {
        const key = cartKey(productId, variantId);
        if (nextQuantity <= 0) state.cart.delete(key);
        else state.cart.set(key, nextQuantity);
        state.activeVariantByProduct[productId] = variantId;
        renderProducts();
      }

      function quickAdd(productId) {
        const product = getProduct(productId);
        const option = selectedVariant(product);
        setQuantity(product.id, option.id, quantity(product.id, option.id) + 1);
        showToast(`${option.label} added to cart`);
      }

      function openOptions(productId) {
        const product = getProduct(productId);
        state.selectedProductId = product.id;
        state.selectedVariantId = state.activeVariantByProduct[product.id] || product.options[0].id;

        sheet.innerHTML = `
          <div class="sheet-body">
            <div class="sheet-handle"></div>
            <header class="sheet-header">
              <div><h2 id="sheetTitle" class="sheet-title">Select an option</h2><p class="sheet-product">${product.name}</p></div>
              <button type="button" class="close-sheet" data-action="close-sheet" aria-label="Close options">×</button>
            </header>
            <div class="option-list" role="radiogroup" aria-label="Product options">
              ${product.options.map(option => `
                <label class="option-row">
                  <input type="radio" name="product-option" value="${option.id}" ${option.id === state.selectedVariantId ? 'checked' : ''}>
                  <span><span class="option-size">${option.label}</span><span class="option-meta">${option.availability} · ${option.discount}</span></span>
                  <span class="option-price">${money(option.price)}<del>${money(option.mrp)}</del></span>
                </label>`).join('')}
            </div>
            <button type="button" id="sheetAdd" class="sheet-add">Add to cart</button>
          </div>`;
        sheet.classList.add('is-open');
        scrim.classList.add('is-open');
        sheet.setAttribute('aria-hidden', 'false');
        document.querySelector('#sheetAdd').addEventListener('click', addSelectedOption);
        sheet.querySelectorAll('input[name="product-option"]').forEach(input => {
          input.addEventListener('change', event => { state.selectedVariantId = event.target.value; });
        });
        sheet.querySelector('[data-action="close-sheet"]').focus();
      }

      function closeOptions() {
        sheet.classList.remove('is-open');
        scrim.classList.remove('is-open');
        sheet.setAttribute('aria-hidden', 'true');
      }

      function addSelectedOption() {
        const product = getProduct(state.selectedProductId);
        const option = getVariant(product, state.selectedVariantId);
        // Different selected variants use different Map keys and therefore remain separate cart items.
        setQuantity(product.id, option.id, quantity(product.id, option.id) + 1);
        closeOptions();
        showToast(`${option.label} added to cart`);
      }

      productGrid.addEventListener('click', event => {
        const button = event.target.closest('button[data-action]');
        if (!button) return;
        const { action, productId, variantId } = button.dataset;
        if (action === 'options') openOptions(productId);
        if (action === 'quick-add') quickAdd(productId);
        if (action === 'increment') setQuantity(productId, variantId, quantity(productId, variantId) + 1);
        if (action === 'decrement') setQuantity(productId, variantId, quantity(productId, variantId) - 1);
        if (action === 'favourite') {
          button.classList.toggle('is-favourite');
          button.textContent = button.classList.contains('is-favourite') ? '♥' : '♡';
        }
      });

      sheet.addEventListener('click', event => {
        if (event.target.closest('[data-action="close-sheet"]')) closeOptions();
      });
      scrim.addEventListener('click', closeOptions);
      document.addEventListener('keydown', event => { if (event.key === 'Escape' && sheet.classList.contains('is-open')) closeOptions(); });

      renderProducts();
    </script>
  </body>
  </html>
```